

Namespace Fl.Event.Handling

Classes

[ExceptionCatcherEventProcessor<TEvent>](#)

An [IEventProcessor<TEvent>](#) decorator that catches any unhandled exceptions thrown by the inner processor and converts them into LanguageExt.Common.Error values.

[LoggingEventProcessor<TEvent>](#)

An [IEventProcessor<TEvent>](#) decorator that adds structured logging around the inner processor's execution using a Serilog [Serilog.ILogger](#).

[ParallelEventProcessor<T>](#)

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances concurrently.

[SequentialEventProcessor<T>](#)

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances one at a time, in order.

Interfaces

[IEventHandler<TEvent>](#)

Defines a handler responsible for processing a single event of type [TEvent](#).

[IEventProcessor<TEvent>](#)

Defines a processor that orchestrates the handling of events of type [TEvent](#).

Class

ExceptionCatcherEventProcessor<TEvent>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

An [IEventProcessor<TEvent>](#) decorator that catches any unhandled exceptions thrown by the inner processor and converts them into LanguageExt.Common.Error values.

```
public class ExceptionCatcherEventProcessor<TEvent> : IEventProcessor<TEvent> where
    TEvent : class
```

Type Parameters

TEvent

The type of the event to process. Must be a reference type.

Inheritance

[object](#)  ← ExceptionCatcherEventProcessor<TEvent>

Implements

[IEventProcessor](#)<TEvent>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Constructors

ExceptionCatcherEventProcessor(IEventProcessor<TEvent>)

An [IEventProcessor<TEvent>](#) decorator that catches any unhandled exceptions thrown by the inner processor and converts them into LanguageExt.Common.Error values.

```
public ExceptionCatcherEventProcessor(IEventProcessor<TEvent> processor)
```

Parameters

`processor` [IEventProcessor<TEvent>](#)

The inner [IEventProcessor<TEvent>](#) to wrap.

Methods

ProcessAsync(TEvent)

Asynchronously processes the specified event, catching any exceptions thrown by the inner processor and returning them as `LanguageExt.Common.Error` values.

```
public EitherAsync<Error, Unit> ProcessAsync(TEvent evt)
```

Parameters

`evt` `TEvent`

The event instance to process.

Returns

`EitherAsync<Error, Unit>`

An `LanguageExt.EitherAsync<L, R>` that resolves to `LanguageExt.Unit` on success, or an `LanguageExt.Common.Error` wrapping the caught exception if processing fails.

Interface IEventHandler<TEvent>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

Defines a handler responsible for processing a single event of type **TEvent**.

```
public interface IEventHandler<TEvent> where TEvent : class
```

Type Parameters

TEvent

The type of the event to handle. Must be a reference type.

Methods

HandleAsync(TEvent)

Asynchronously handles the specified event.

```
EitherAsync<Error, Unit> HandleAsync(TEvent evt)
```

Parameters

evt TEvent

The event instance to handle.

Returns

EitherAsync<Error, Unit>

An LanguageExt.EitherAsync<L, R> that resolves to LanguageExt.Unit on success, or an LanguageExt.Common.Error if handling fails.

Interface IEventProcessor<TEvent>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

Defines a processor that orchestrates the handling of events of type **TEvent**.

```
public interface IEventProcessor<TEvent> where TEvent : class
```

Type Parameters

TEvent

The type of the event to process. Must be a reference type.

Methods

ProcessAsync(TEvent)

Asynchronously processes the specified event.

```
EitherAsync<Error, Unit> ProcessAsync(TEvent evt)
```

Parameters

evt TEvent

The event instance to process.

Returns

EitherAsync<Error, Unit>

An LanguageExt.EitherAsync<L, R> that resolves to LanguageExt.Unit on success, or an LanguageExt.Common.Error if processing fails.

Class LoggingEventProcessor<TEvent>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

An [IEventProcessor<TEvent>](#) decorator that adds structured logging around the inner processor's execution using a Serilog [Serilog.ILogger](#).

```
public class LoggingEventProcessor<TEvent> : IEventProcessor<TEvent> where TEvent : class
```

Type Parameters

TEvent

The type of the event to process. Must be a reference type.

Inheritance

[object](#)  ← LoggingEventProcessor<TEvent>

Implements

[IEventProcessor](#)<TEvent>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

LoggingEventProcessor(IEventProcessor<TEvent>, ILogger)

An [IEventProcessor<TEvent>](#) decorator that adds structured logging around the inner processor's execution using a Serilog [Serilog.ILogger](#).

```
public LoggingEventProcessor(IEventProcessor<TEvent> processor, ILogger logger)
```

Parameters

processor [IEventProcessor<TEvent>](#)

The inner [IEventProcessor<TEvent>](#) to wrap.

logger [ILogger](#)

The Serilog logger used to record processing outcomes.

Methods

ProcessAsync(TEvent)

Asynchronously processes the specified event and logs the outcome via the configured [Serilog.ILogger](#).

```
public EitherAsync<Error, Unit> ProcessAsync(TEvent evt)
```

Parameters

evt [TEvent](#)

The event instance to process.

Returns

[EitherAsync<Error, Unit>](#)

An [LanguageExt.EitherAsync<L, R>](#) that resolves to [LanguageExt.Unit](#) on success, or an [LanguageExt.Common.Error](#) if processing fails.

Class ParallelEventProcessor<T>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances concurrently.

```
public class ParallelEventProcessor<T> : IEventProcessor<T> where T : class
```

Type Parameters

T

The type of the event to process. Must be a reference type.

Inheritance

[object](#)  ← ParallelEventProcessor<T>

Implements

[IEventProcessor](#)<T>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

ParallelEventProcessor(IEnumerable<IEventHandler<T>>)

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances concurrently.

```
public ParallelEventProcessor(IEnumerable<IEventHandler<T>> eventHandlers)
```

Parameters

`eventHandlers` [IEnumerable](#) <[IEventHandler](#)<T>>

The collection of handlers to invoke in parallel.

Methods

ProcessAsync(T)

Asynchronously dispatches the specified event to all registered handlers in parallel, awaiting all of them before returning.

```
public EitherAsync<Error, Unit> ProcessAsync(T evt)
```

Parameters

`evt` T

The event instance to process.

Returns

`EitherAsync<Error, Unit>`

An `LanguageExt.EitherAsync<L, R>` that resolves to `LanguageExt.Unit` when all handlers succeed, or an `LanguageExt.Common.Error` if any handler fails.

Class SequentialEventProcessor<T>

Namespace: [Fl.Event.Handling](#)

Assembly: Fl.Event.Handling.dll

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances one at a time, in order.

```
public class SequentialEventProcessor<T> : IEventProcessor<T> where T : class
```

Type Parameters

T

The type of the event to process. Must be a reference type.

Inheritance

[object](#)  ← SequentialEventProcessor<T>

Implements

[IEventProcessor](#)<T>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

SequentialEventProcessor(IEnumerable<IEventHandler<T>>)

An [IEventProcessor<TEvent>](#) that dispatches an event to all registered [IEventHandler<TEvent>](#) instances one at a time, in order.

```
public SequentialEventProcessor(IEnumerable<IEventHandler<T>> eventHandlers)
```

Parameters

`eventHandlers` [IEnumerable](#) <[IEventHandler](#)<T>>

The ordered collection of handlers to invoke sequentially.

Methods

ProcessAsync(T)

Asynchronously dispatches the specified event to all registered handlers one at a time, stopping early if any handler returns an `LanguageExt.Common.Error`.

```
public EitherAsync<Error, Unit> ProcessAsync(T evt)
```

Parameters

`evt` T

The event instance to process.

Returns

`EitherAsync<Error, Unit>`

An `LanguageExt.EitherAsync<L, R>` that resolves to `LanguageExt.Unit` when all handlers succeed, or an `LanguageExt.Common.Error` if any handler fails.

Namespace Fl.Event.Handling.Extensions

Classes

[ServiceCollectionExtensions](#)

Provides extension methods for registering event handling services with the [IServiceCollection](#) dependency injection container.

Enums

[EventHandlerStrategy](#)

Specifies the strategy used to dispatch events to their registered handlers.

Enum EventHandlingStrategy

Namespace: [Fl.Event.Handling.Extensions](#)

Assembly: Fl.Event.Handling.Extensions.dll

Specifies the strategy used to dispatch events to their registered handlers.

```
public enum EventHandlingStrategy
```

Fields

`Parallel = 1`

Handlers are invoked concurrently and all are awaited before the result is returned.

`Sequential = 0`

Handlers are invoked one at a time, in registration order. Processing stops immediately if any handler returns an error.

Class ServiceCollectionExtensions

Namespace: [Fl.Event.Handling.Extensions](#)

Assembly: Fl.Event.Handling.Extensions.dll

Provides extension methods for registering event handling services with the [IServiceCollection](#) dependency injection container.

```
public static class ServiceCollectionExtensions
```

Inheritance

[object](#) ← ServiceCollectionExtensions

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

AddEventHandling<TEvent>(IServiceCollection, EventHandlingStrategy, Assembly)

Registers an [IEventProcessor<TEvent>](#) and all [IEventHandler<TEvent>](#) implementations found in the specified `assembly` with the DI container.

```
public static IServiceCollection AddEventHandling<TEvent>(this IServiceCollection  
    services, EventHandlingStrategy strategy, Assembly assembly) where TEvent : class
```

Parameters

`services` [IServiceCollection](#)

The [IServiceCollection](#) to add the services to.

`strategy` [EventHandlingStrategy](#)

The [EventHandlerStrategy](#) that determines whether handlers are invoked sequentially or in parallel.

assembly [Assembly](#)

The [Assembly](#) to scan for [IEventHandler<TEvent>](#) implementations.

Returns

[IServiceCollection](#)

The original `services` instance for method chaining.

Type Parameters

TEvent

The event type to configure handling for. Must be a reference type.

Exceptions

[ArgumentException](#)

Thrown when `strategy` is not a recognised [EventHandlerStrategy](#) value.